

# i-mSEC-AT

## LLM Prompt Catalogue

Structured overview of the centrally managed prompts used for requirement translation, evidence-grounded justification drafting, and audit-summary narrative synthesis

|                  |                                                                                                                                              |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Document purpose | Reference catalogue for the versioned runtime LLM prompts used by the i-mSEC-AT audit workflow                                               |
| Scope            | Prompt catalogue structure, loader and provenance, invocation configuration, runtime inputs, output contracts, and complete prompt templates |
| Prompt source    | scripts/llm_prompts.yml, loaded and validated by scripts/prompt_loader.py                                                                    |
| Prepared date    | 15 September 2026                                                                                                                            |

## Contents

---

|                                                  |    |
|--------------------------------------------------|----|
| Overview of the LLM prompts used in i-mSEC-AT    | 3  |
| Central prompt catalogue, loader, and provenance | 4  |
| Model configuration and invocation behaviour     | 5  |
| 1. English translation prompt                    | 6  |
| 2. Requirement justification prompt              | 8  |
| 3. Audit summary narrative prompt                | 11 |

## Overview of the LLM prompts used in i-mSEC-AT

The current i-mSEC-AT implementation defines three runtime LLM prompts. All three are centralised in `scripts/llm_prompts.yml` and are retrieved through `scripts/prompt_loader.py` rather than being embedded as prompt text inside the calling Python modules. The catalogue is versioned and its raw contents are hashed so that generated artefacts can record the exact prompt set used at runtime.

The first prompt translates requirement descriptions into English when non-English catalogue text is detected. The second prompt drafts evidence-grounded English justifications for outcomes that have already been computed by deterministic requirement-evaluation logic. The third prompt supports audit-summary reporting by improving weakness-pattern prose and producing grounded recommendations from workbook-derived counts and evidence anchors.

A material architectural distinction from the previous catalogue is that there is no requirement-level LLM classification prompt in the current runtime prompt set. Compliance outcomes are computed in code from mapped fingerprint flags, applicability conditions, out-of-scope capabilities, and deterministic precedence rules. The LLM may explain a precomputed result, but it is explicitly instructed not to change that result.

This arrangement separates assessment from language generation. Security-relevant decisions remain traceable to deterministic evidence processing, while the LLM is used for translation and controlled narrative generation. Each LLM-facing task is constrained to JSON-as-text inputs and structured JSON outputs, with deterministic fallbacks where applicable.

### Prompt inventory

| # | Prompt ID                 | Invoked by                              | Role in the pipeline                                                            | Output                             |
|---|---------------------------|-----------------------------------------|---------------------------------------------------------------------------------|------------------------------------|
| 1 | requirement_translation   | ai_security_audit_requirements_excel.py | Translate non-English requirement descriptions to English.                      | items[id, text_en]                 |
| 2 | requirement_justification | ai_security_audit_requirements_excel.py | Draft wording for precomputed requirement outcomes without altering the result. | items[id, justification]           |
| 3 | audit_summary_narrative   | audit_summary_stage2_generate_docx.py   | Improve weakness-pattern prose and generate grounded recommendations.           | key_takeaways;<br>pattern_writeups |

#### Assessment boundary

Deterministic assessment rules remain in Python code. The YAML catalogue contains only the instructions and payload contracts that are sent to an LLM.

## Central prompt catalogue, loader, and provenance

The prompt architecture uses a single catalogue file, `scripts/llm_prompts.yml`. At catalogue level, the current file declares `schema_version "1.0"`, `prompt_set_version "1.0.0"`, and `default_language "en"`. Each prompt entry carries its own version, pipeline stage, purpose, system prompt, and user-payload contract. Output contracts are also declared where the caller uses a simple item schema; the audit-summary entry instead embeds its required output schema inside the user payload.

### Validated loading behaviour

| Component                           | Current behaviour                                                                                                                   | Traceability value                                                            |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| PROMPTS_PATH                        | Resolved beside <code>prompt_loader.py</code> as <code>scripts/llm_prompts.yml</code> .                                             | Avoids dependence on the process working directory.                           |
| <code>load_prompt_catalog()</code>  | Reads raw bytes, parses YAML with <code>yaml.safe_load</code> , and validates catalogue and prompt fields.                          | Rejects malformed or incomplete prompt definitions before invocation.         |
| Required prompt fields              | <code>version</code> , <code>pipeline_stage</code> , <code>purpose</code> , <code>system_prompt</code> , <code>user_prompt</code> . | Provides a stable minimum contract for every runtime prompt.                  |
| Catalogue hash                      | SHA-256 is calculated over the raw YAML bytes and attached to the loaded catalogue.                                                 | Identifies the exact prompt catalogue content used for an artefact.           |
| <code>load_prompt(prompt_id)</code> | Returns <code>copy.deepcopy(prompt)</code> .                                                                                        | Allows callers to inject runtime data without mutating the shared definition. |
| <code>prompt_provenance()</code>    | Returns schema version, prompt-set version, catalogue SHA-256, and each used prompt ID/version.                                     | Supports reproducible audit artefacts and change tracking.                    |

### Runtime flow

```
Caller script
-> load_prompt(prompt_id)
-> load_prompt_catalog() validates scripts/llm_prompts.yml
-> isolated prompt definition returned
-> caller injects runtime JSON data
-> OpenAI-compatible Responses API invocation
-> structured JSON parsing / validation or JSON extraction
-> deterministic downstream processing or fallback
-> prompt_provenance(...) records prompt versions and catalogue SHA-256
```

### Provenance artefacts

The `requirement-audit` script writes `/mnt/data/requirement_prompt_provenance.json` after producing `security_audit_requirements.xlsx`. The summary generator writes `/mnt/data/audit_summary_prompt_provenance.json` after producing the audit-summary DOCX. Both provenance files include the prompt catalogue identity and the model identifier recorded from `LLM_MODEL`; the requirement-audit provenance also records relevant configuration switches.

## Model configuration and invocation behaviour

The calling scripts use the OpenAI Python SDK against an OpenAI-compatible endpoint. This allows the same prompt catalogue to be used with hosted or locally deployed backends that implement the compatible Responses API surface, provided the configured model supports the required request parameters and JSON-producing behaviour.

### Shared environment configuration

|                       |                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------|
| Model                 | Environment variable: LLM_MODEL                                                     |
| API key               | Environment variable: LLM_API_KEY                                                   |
| API endpoint          | Environment variable: LLM_BASE_URL                                                  |
| Reasoning effort      | Environment variable: LLM_REASONING_EFFORT; default: medium                         |
| Maximum output tokens | Environment variable: LLM_MAX_OUTPUT_TOKENS; caller-specific defaults               |
| Input transport       | System message plus compact JSON-as-text user payload; no OpenAI file input is used |

### Call-path differences

| Prompt(s)                   | Invocation method                                                          | Default max output | Temperature                                            | Failure behaviour                                                                                                                    |
|-----------------------------|----------------------------------------------------------------------------|--------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Translation + justification | client.responses.parse when available;<br>client.responses.create fallback | 2000 tokens        | 0 on create fallback; not explicitly set on parse path | Up to three attempts. Translation may become fatal under strict-English conditions; justification falls back to deterministic prose. |
| Audit summary narrative     | client.responses.create                                                    | 6000 tokens        | 0                                                      | Caller catches exceptions and continues with non-LLM report wording/fallback content.                                                |

### Invocation conditions

Requirement translation is invoked only when non-English requirement descriptions are detected and an LLM client is available. Requirement justification is optional and is invoked only when USE\_OPENAI\_JUSTIFICATIONS is enabled; otherwise the script uses deterministic\_justification. Audit-summary narrative enhancement is attempted only when the OpenAI client is importable and LLM\_API\_KEY is configured. Consequently, the assessment and report pipeline can retain deterministic behaviour even when narrative enhancement is unavailable.

#### Structured-output policy

The first two tasks prefer Pydantic-backed Responses API parsing when supported. Their fallback path extracts a terminal JSON object from output\_text. The audit-summary task always extracts and parses a JSON object from output\_text.

## 1. English translation prompt

Catalogue entry: `scripts/llm_prompts.yml -> prompts.requirement_translation` | Invoked by: `scripts/ai_security_audit_requirements_excel.py`

### LLM invocation configuration

|                       |                                                               |
|-----------------------|---------------------------------------------------------------|
| Prompt version        | 1.0.0                                                         |
| Pipeline stage        | requirement assessment                                        |
| Invocation method     | client.responses.parse, with client.responses.create fallback |
| Input format          | System prompt + JSON user payload                             |
| Model                 | LLM_MODEL                                                     |
| API key / endpoint    | LLM_API_KEY / LLM_BASE_URL                                    |
| Reasoning effort      | LLM_REASONING_EFFORT; default medium                          |
| Maximum output tokens | LLM_MAX_OUTPUT_TOKENS; default 2000                           |
| Output contract       | Strict JSON: items[id, text_en]                               |

This prompt is used when the requirements catalogue contains descriptions that appear to be non-English. The caller sends a compact batch containing requirement identifiers and source text. The model is restricted to English translation and is not permitted to add requirements or expand the substantive meaning of the source description.

Under the default strict-English behaviour, a detected non-English description requires a successful translation before the final audit workbook can be generated. If translation is needed but no LLM client is available, the caller terminates rather than silently emitting non-English generated artefacts.

#### Main role

Provide concise professional English descriptions while preserving the original requirement semantics and a stable identifier-to-translation mapping.

## Input parameters

| Parameter | Runtime source                            | Purpose / meaning                                                                  |
|-----------|-------------------------------------------|------------------------------------------------------------------------------------|
| items     | Translation batch assembled by the caller | List of objects to be translated.                                                  |
| id        | Requirement object / PUID                 | Stable identifier used to map the returned translation to the correct requirement. |
| text      | Original requirement description          | Source description supplied to the model for translation.                          |

## System prompt

You translate short requirement descriptions to English.  
Strict rules:  
- Output English only.  
- Preserve meaning. Do not add new requirements.  
- Keep the translation concise and professional.  
- Return ONLY JSON in the form: {"items": [{"id": "...", "text\_en": "..."}, ...]}.

## Runtime user payload

```
{
  "items": [
    {
      "id": "{puid}",
      "text": "{requirement_description}"
    }
  ]
}
```

## Expected response

```
{
  "items": [
    {
      "id": "{puid}",
      "text_en": "<concise English translation>"
    }
  ]
}
```

*The caller records requirement\_translation in PROMPTS\_INVOKED only when this prompt is actually loaded and invoked.*

## 2. Requirement justification prompt

Catalogue entry: `scripts/llm_prompts.yml -> prompts.requirement_justification` | Invoked by: `scripts/ai_security_audit_requirements_excel.py`

### LLM invocation configuration

|                       |                                                               |
|-----------------------|---------------------------------------------------------------|
| Prompt version        | 1.0.0                                                         |
| Pipeline stage        | requirement assessment                                        |
| Invocation method     | client.responses.parse, with client.responses.create fallback |
| Activation            | Optional; USE_OPENAI_JUSTIFICATIONS must be enabled           |
| Input format          | System prompt + JSON user payload                             |
| Reasoning effort      | LLM_REASONING_EFFORT; default medium                          |
| Maximum output tokens | LLM_MAX_OUTPUT_TOKENS; default 2000                           |
| Output contract       | Strict JSON: items[id, justification]                         |

This prompt drafts concise English justifications for requirement outcomes that have already been computed by the deterministic i-mSEC-AT evaluation logic. The caller first evaluates each requirement against mapped VISION360 fingerprint flags and produces result, determination\_basis, applicability metadata, and flag-level evidence. Only then is the prompt given the context needed to explain that outcome.

The prompt is therefore explanatory rather than decisional. It explicitly forbids changing the precomputed result and requires the generated wording to preserve meta.determination\_basis. This distinction prevents missing or ambiguous evidence from being rhetorically converted into an evidenced contradiction.

### Deterministic assessment boundary

#### No LLM classification step

The current runtime catalogue contains no prompt that decides YES/NO/N/A. `audit_requirement(...)` computes yes/no/n/a in code from flag classification, expected values, applicability gates, out-of-scope capabilities, evidence counts, and conservative precedence rules. If LLM justification is disabled or fails, `deterministic_justification(...)` is used instead.

The iOS-specific N/A handling is also encoded in the prompt contract. When `meta.out_of_scope_capabilities` is non-empty, the justification must name every capability and explain that `runtime_device_test`, `backend_evidence`, `organizational_evidence`, and `manual_review` fall outside the automated static iOS audit profile, while `signed_ipa` indicates that only an unsigned IPA was supplied. These conditions are not to be described as security failures.

## Input parameters

| Parameter             | Source                                     | Purpose / meaning                                                                                  |
|-----------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------|
| id                    | Requirement audit context                  | Identifier of the requirement being justified.                                                     |
| description_en        | Translated or original English description | English requirement text used in the audit output.                                                 |
| result                | Deterministic assessment logic             | Precomputed requirement result: yes, no, or n/a.                                                   |
| flags_used            | Requirement-to-flag mapping                | Fingerprint flag identifiers mapped to the requirement.                                            |
| meta                  | Deterministic assessment metadata          | Includes prohibitive/conditional state, gates, out_of_scope_capabilities, and determination_basis. |
| flags[].id            | Flag evidence object                       | Identifier of the mapped fingerprint flag.                                                         |
| classification        | Requirement-aware flag classification      | POSITIVE_CONTROL, NEGATIVE_RISK, or APPLICABILITY.                                                 |
| expected              | Deterministic expectation                  | YES for a positive control, NO for a negative risk, or null for applicability.                     |
| observed_summary_norm | Normalised fingerprint summary             | YES, NO, or NA parsed from app_verdict.summary.                                                    |
| app_verdict.state     | Fingerprint app_verdict                    | Original verdict state, including out_of_scope or missing states.                                  |
| app_verdict.summary   | Fingerprint app_verdict                    | Original summary string used to derive the normalised observation.                                 |
| app_verdict.notes     | Fingerprint app_verdict                    | Notes sent in bounded form; non-English notes may be replaced with a paraphrase instruction.       |
| evidence_count        | Fingerprint app_verdict                    | Number of evidence items associated with the flag.                                                 |
| note_hint             | Derived by caller                          | Set to 'Fallback verdict' when that phrase appears in notes.                                       |
| outcome               | Evaluation against expectation             | SUPPORT, CONTRADICT, UNKNOWN, MISSING, NA_OR_GATE, or OUT_OF_SCOPE as applicable.                  |

*The batch size defaults to 25 requirements. Notes are truncated before being sent to the LLM, and strict-English mode prevents non-English notes from being quoted verbatim.*

## System prompt

You draft audit justifications in English for security requirement outcomes.

Strict rules:

- Output English only.
- If any provided notes contain non-English text, paraphrase them into English and do not quote them verbatim.
- Do not invent evidence or flags.
- Use only the provided context.
- Keep 1 to 3 sentences per requirement.
- Explicitly mention: `app_verdict.state`, normalized `app_verdict.summary` (YES/NO/NA), notes (include 'Fallback verdict' if present), and `evidence_count`.
- If a flag is not present in the fingerprint, state: 'flag not present in fingerprint'.
- For an n/a result, explicitly state every capability in `meta.out_of_scope_capabilities`. Explain that `runtime_device_test`, `backend_evidence`, `organizational_evidence`, and `manual_review` are outside the automated static iOS audit profile, and that `signed_ipa` means only an unsigned IPA was supplied. Do not describe any out-of-scope case as a security failure.
- Do not change the precomputed result.
- Preserve and explain `meta.determination_basis`; do not present insufficient evidence as an evidenced contradiction.
- Return ONLY JSON in the form: `{ "items": [ { "id": "...", "justification": "...", ... } ] }`.

## Runtime user payload

```
{
  "batch": [
    {
      "id": "{puid}",
      "description_en": "{description_en}",
      "result": "{result}",
      "flags_used": [ "{flag_id}", "..."],
      "meta": { "determination_basis": "...", "out_of_scope_capabilities": ["..."] },
      "flags": [
        {
          "id": "{flag_id}",
          "classification": "{classification}",
          "expected": "{expected}",
          "observed_summary_norm": "{YES|NO|NA}",
          "app_verdict": {
            "state": "{state}", "summary": "{summary}",
            "notes": "{notes}", "evidence_count": "{evidence_count}",
            "note_hint": "{note_hint}"
          },
          "outcome": "{outcome}"
        }
      ]
    }
  ]
}
```

### 3. Audit summary narrative prompt

Catalogue entry: `scripts/llm_prompts.yml -> prompts.audit_summary_narrative` | Invoked by: `scripts/audit_summary_stage2_generate_docx.py`

#### LLM invocation configuration

|                       |                                                                             |
|-----------------------|-----------------------------------------------------------------------------|
| Prompt version        | 1.0.0                                                                       |
| Pipeline stage        | narrative synthesis and reporting                                           |
| Invocation method     | client.responses.create                                                     |
| Activation            | Attempted when the OpenAI client is available and LLM_API_KEY is configured |
| Input format          | System prompt + JSON user payload derived from the catalogue entry          |
| Reasoning effort      | LLM_REASONING_EFFORT; default medium                                        |
| Maximum output tokens | LLM_MAX_OUTPUT_TOKENS; default 6000                                         |
| Temperature           | 0                                                                           |
| Expected output       | Strict JSON with key_takeaways and pattern_writeups                         |

This prompt supports the narrative-synthesis stage of audit-summary generation. It is not used to classify individual requirements and it does not produce audit metrics. The caller receives weakness patterns and prevalence data from the stage-1 analysis pack, normalises the top patterns for the LLM, and asks the model to improve wording and provide practical recommendations.

The prompt is deliberately bounded by workbook-derived information. It forbids invented implemented controls, invented metrics, and factual claims beyond the supplied anchors and counts. The caller passes at most the first ten weakness patterns and at most two description anchors for each pattern. The resulting prose is then integrated into the report only as a narrative enhancement layer.

#### Main role

Transform already-derived weakness-pattern evidence into clear key takeaways, expected-control descriptions, impact statements, and practical recommendations without altering the underlying counts or inventing evidence.

## Input parameters

| Parameter         | Runtime source                      | Purpose / meaning                                                              |
|-------------------|-------------------------------------|--------------------------------------------------------------------------------|
| max_key_takeaways | Function argument; caller uses 7    | Upper bound communicated to the model for the takeaway set.                    |
| prevalence_rubric | Audit analysis pack                 | Mapping that explains how prevalence labels relate to workbook-derived counts. |
| pattern           | Weakness-pattern record             | Exact pattern name that must be preserved in pattern_writeups.                 |
| count             | mapped_noncompliant_count           | Number of non-compliant controls mapped to the weakness pattern.               |
| severity          | Weakness-pattern record             | Severity associated with the pattern.                                          |
| prevalence        | Derived by _prevalence_from_count() | Prevalence label calculated from the count before LLM invocation.              |
| owner             | recommended_owner                   | Team or role associated with remediation ownership.                            |
| anchors           | description_anchors[:2]             | Short evidence anchors that constrain the prose.                               |

## System prompt

You are a senior security audit reporting specialist. You will improve wording and generate actionable recommendations ONLY at the weakness-pattern level. You must NOT invent specific implemented controls. You must NOT invent metrics. You must NOT claim facts beyond the provided anchors and counts. Return strict JSON only.

## Caller-side runtime normalisation

```
{
  "pattern": p["pattern"],
  "count": int(p["mapped_noncompliant_count"]),
  "severity": p.get("severity", "Low"),
  "prevalence": _prevalence_from_count(count),
  "owner": p.get("recommended_owner", "Engineering"),
  "anchors": p.get("description_anchors", [])[:2]
}
```

## User payload contract

```
{
  "task": "Generate paper-quality prose components grounded in workbook-derived prevalence counts.",
  "constraints": {
    "no_invented_metrics": true,
    "no_category_level_bullet_dumps": true,
    "no_long_id_lists": true,
    "recommendations_no_time_headings": true,
    "max_key_takeaways": "<runtime: integer>",
    "prevalence_rubric": "<runtime: mapping>"
  },
  "input_patterns": "<runtime: normalized weakness-pattern evidence>",
  "required_output_schema": {
    "key_takeaways": [
      "<5-7 bullets; each references prevalence count and pattern name>"
    ],
    "pattern_writeups": [
      {
        "pattern": "<exact pattern name from input>",
        "expected": "<1-2 sentences>",
        "impact": "<1-2 sentences; CIA + privacy/regulatory context where relevant>",
        "recommendations": [
          "<6-10 bullets; practical and specific to this pattern; do not recommend authentication controls for unrelated patterns>"
        ]
      }
    ]
  }
}
```

## Fallback and traceability behaviour

If the narrative LLM call is not attempted or raises an exception, audit-summary generation continues. The caller can construct default key-takeaway text directly from each pattern name, severity, and mapped non-compliance count, and the remainder of the DOCX is still produced from the deterministic analysis pack.

After report generation, `prompt_provenance(sorted(PROMPTS_INVOKED))` records the exact prompt ID/version set and the SHA-256 of `scripts/llm_prompts.yml` in `/mnt/data/audit_summary_prompt_provenance.json`. This makes prompt changes independently auditable from the generated report content.

### Catalogue maintenance rule

Prompt wording should be changed in `scripts/llm_prompts.yml`. Calling modules should continue to refer to stable prompt IDs through `load_prompt(...)`, leaving deterministic assessment logic in code and preserving provenance through `prompt_provenance(...)`.